

Problem Formulation – Recommendation Data Pipeline

Group assignment – MTech (Data Engineering), BITS Pilani WILP

1. Business Problem

RecoMart is an online shopping platform. Most users only see a few popular products because they mostly use search, and they do not explore the rest of the catalog. This is a problem because it keeps conversion rate low and stops repeat visits.

The business problem we are solving is: show each user the products they are most likely to click, add to cart, or buy, based on their own past activity and the activity of similar users.

If this works, we expect two things to improve: conversion rate (how many browsing users actually buy) and click rate on the recommended products. We also want good catalog coverage, meaning the system should not just keep recommending the same best sellers to everyone.

2. Data Sources

We are using two data sources. One gives us user behaviour, and the other gives us user profile data.

2.1 REES46 eCommerce Behaviour Dataset (CSV)

This is a public dataset of real online store activity, published by REES46. We downloaded one month of data (CSV file) from the multi-category store. It has one row for every action a user takes on the website.

Column	What it means
event_time	date and time when the action happened
event_type	type of action: view, cart, or purchase
product_id	id of the product
category_id / category_code	which category the product belongs to, e.g. electronics.smartphone
brand	brand name of the product
price	price of the product
user_id	id of the user who did the action
user_session	id of the browsing session

This file is our main source for building implicit feedback. A view is a weak signal, add to cart is a medium signal, and a purchase is a strong signal. These three together tell us what a user actually likes, even without asking them to rate anything directly.

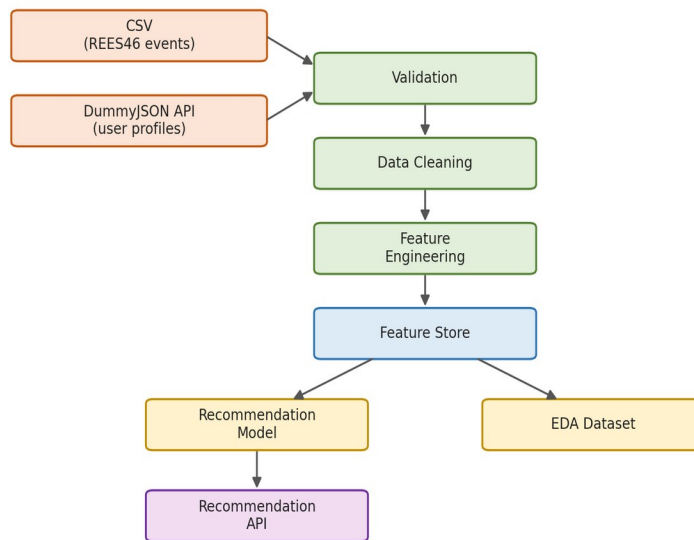
2.2 DummyJSON API

The CSV file does not have any information about the user, like age, gender or city. So we are using the DummyJSON API to get sample user profile data, and joining it to the user_id from the CSV. DummyJSON is used as a representative source for customer profile data in this project. It also lets us show API based ingestion in the pipeline without needing access to any real customer data.

Endpoint	Attributes we use
/users	id, age, gender, city – used as user features, also useful for cold start users

3. Pipeline Architecture

The diagram below shows how data moves through the pipeline, from the two raw sources to the final model and API.



Each stage (ingestion, validation, transformation, feature engineering, model training, and inference) is implemented as an independent module. This makes the pipeline easier to test, maintain, and extend.

4. Data Validation and Data Quality

The report mentions validation as a stage, so this section explains what is actually checked at that stage.

Validation	Example
Missing values	Brand cannot be NULL after enrichment
Duplicate events	Remove duplicate user-product events
Invalid prices	Price must be greater than 0
Invalid timestamps	Ignore future timestamps
Schema validation	Required columns must exist

Data quality checks ensure missing values, duplicate events, invalid timestamps, and incorrect prices are detected before downstream processing. This prevents poor-quality data from affecting recommendations.

5. Batch vs Near Real-Time Processing

The pipeline needs to support both batch and near real-time ingestion, so these two are kept separate with different purposes.

Processing Type	Purpose
Batch	Daily model retraining, using the full historical data
Near Real-Time	Update user features right after every interaction, so recent activity is reflected quickly

Since the CSV is historical data, we do not have an actual live stream. To still demonstrate the near real-time part, we plan to replay the CSV rows in time order, as if they are arriving live, and update the features as each new event comes in.

6. Technology Stack

The pipeline will be implemented using Apache Airflow for orchestration, Apache Spark (or Pandas for smaller data) for processing, PostgreSQL as the feature store, FastAPI for inference, and Docker for deployment.

Apache Airflow schedules and monitors each stage of the pipeline, so the batch workflows run in the correct order and we can see where a run failed if something breaks.

7. Expected Outputs From the Pipeline

The pipeline should not just give us one final model. It should produce a few separate outputs, each used by a different part of the project:

- Clean datasets for EDA – the raw CSV has missing brand and category values and duplicate rows, so this step removes duplicates, fixes data types, and handles missing values. This cleaned data is what we use for exploring and understanding the data before building anything.
- Engineered features for the models, for example:
 - User features – purchase frequency, average basket size, favorite category, brand affinity
 - Product features – popularity score, average rating, category encoding, price bucket
 - Interaction features – weighted interaction score, recency score
- A deployable recommendation model – a hybrid model combining collaborative filtering and content based filtering, saved in a way that it can be loaded again and reused, not just run once in a notebook.
- An inference interface – a simple API where you give a user_id and it returns a list of recommended products. This is what the actual website would call.

8. Evaluation Metrics

This is a ranking problem, we are not predicting one label, we are predicting an ordered list of products for each user. So normal accuracy does not make sense here, we use ranking metrics instead. We split the data by time, earlier events for training and later events for testing, so the model is not tested on data it could have seen in training.

Metric	What it tells us
Precision@K	out of the K products we recommended, how many did the user actually interact with
Recall@K	out of all the products the user actually interacted with, how many did we manage to include in our top K
NDCG@K	like precision and recall, but gives more credit if the correct products are ranked near the top of the list
Coverage	how much of the full catalog gets recommended at least once, so we know the model is not just repeating the same popular items

We are using $K = 10$, since that matches a normal size for a “recommended for you” section on a product page. Offline metrics will be evaluated first. In a production environment, online A/B testing would be used to measure improvements in Click Through Rate (CTR) and Conversion Rate.

9. Team Split

The team has five members. Work is divided across ingestion, storage, data validation, feature engineering, feature store, model versioning, model training, and orchestration. Each person owns one part, but we agreed on the table and file formats between each stage in advance so all the parts connect properly.

10. Conclusion

This pipeline combines batch and near-real-time processing to provide fresh, validated, and feature-rich data for recommendation models. The modular architecture improves maintainability while supporting scalable recommendation generation for RecoMart.